3661. Maximum Walls Destroyed by Robots

Solved

Hard | Topics | Companies | Hint

There is an endless straight line populated with some robots and walls. You are given integer arrays `robots`, `distance`, and `walls`:

- `robots[i]` is the position of the i^{th} robot.
- `distance[i]` is the **maximum** distance the i^{th} robot's bullet can travel.
- `walls[j]` is the position of the j^{th} wall.

Every robot has **one** bullet that can either fire to the left or the right **at most** `distance[i]` meters.

A bullet destroys every wall in its path that lies within its range. Robots are fixed obstacles: if a bullet hits another robot before reaching a wall, it **immediately stops** at that robot and cannot continue.

Return the **maximum** number of **unique** walls that can be destroyed by the robots.

Notes:

- A wall and a robot may share the same position; the wall can be destroyed by the robot at that position.
- Robots are not destroyed by bullets.

Example 1:

> **Input:** `robots = [4], distance = [3], walls = [1,10]`
>
> **Output:** 1
>
> **Explanation:**
>
> - `robots[0] = 4` fires **left** with `distance[0] = 3`, covering `[1, 4]` and destroys `walls[0] = 1`.
> - Thus, the answer is 1.

Example 2:

> **Input:** `robots = [10,2], distance = [5,1], walls = [5,2,7]`
>
> **Output:** 3
>
> **Explanation:**
>
> - `robots[0] = 10` fires **left** with `distance[0] = 5`, covering `[5, 10]` and destroys `walls[0] = 5` and `walls[2] = 7`.
> - `robots[1] = 2` fires **left** with `distance[1] = 1`, covering `[1, 2]` and destroys `walls[1] = 2`.
> - Thus, the answer is 3.

Example 3:

> **Input:** `robots = [1,2], distance = [100,1], walls = [10]`
>
> **Output:** `0`
>
> **Explanation:**
>
> In this example, only `robots[0]` can reach the wall, but its shot to the **right** is blocked by `robots[1]`; thus the answer is 0.

Constraints:

- `1 <= robots.length == distance.length <= 10^5`
- `1 <= walls.length <= 10^5`
- `1 <= robots[i], walls[j] <= 10^9`
- `1 <= distance[i] <= 10^5`
- All values in `robots` are **unique**
- All values in `walls` are **unique**

Python:

```
class Solution:
    def maxWalls(self, robots: List[int], distance: List[int], walls: List[int]) -> int:
        n = len(robots)

        # Store robots as [position, distance]
        x = [[robots[i], distance[i]] for i in range(n)]

        # Sort robots and walls
        x.sort()
        walls.sort()

        # Dummy robot
        x.append([10**9, 0])

        # Function to count walls in range [l, r]
        def query(l, r):
            if l > r:
                return 0
            return bisect.bisect_right(walls, r) - bisect.bisect_left(walls, l)
```

```
# dp[i][0] = shoot LEFT
# dp[i][1] = shoot RIGHT
dp = [[0, 0] for _ in range(n)]

# Base case
dp[0][0] = query(x[0][0] - x[0][1], x[0][0])

if n > 1:
    dp[0][1] = query(
        x[0][0],
        min(x[1][0] - 1, x[0][0] + x[0][1])
    )
else:
    dp[0][1] = query(x[0][0], x[0][0] + x[0][1])

# DP transitions
for i in range(1, n):

    # Case 1: shoot RIGHT
    dp[i][1] = max(dp[i - 1][0], dp[i - 1][1]) + \
               query(
                   x[i][0],
                   min(x[i + 1][0] - 1, x[i][0] + x[i][1])
               )

    # Case 2: shoot LEFT (no overlap)
    dp[i][0] = dp[i - 1][0] + \
               query(
                   max(x[i][0] - x[i][1], x[i - 1][0] + 1),
                   x[i][0]
               )

    # Case 3: shoot LEFT with overlap handling
    leftStart = max(x[i][0] - x[i][1], x[i - 1][0] + 1)
    leftEnd = x[i][0]

    overlapStart = leftStart
    overlapEnd = min(x[i - 1][0] + x[i - 1][1], x[i][0] - 1)

    res = dp[i - 1][1] + \
          query(leftStart, leftEnd) - \
          query(overlapStart, overlapEnd)
```

```
            dp[i][0] = max(dp[i][0], res)

        return max(dp[n - 1][0], dp[n - 1][1])
```

JavaScript:

```
/**
 * @param {number[]} robots
 * @param {number[]} distance
 * @param {number[]} walls
 * @return {number}
 */
var maxWalls = function(robots, distance, walls) {
    let n = robots.length;
    let r = [];
    let rs = new Set();

    for (let i = 0; i < n; i++) {
        r.push([robots[i], distance[i]]);
        rs.add(robots[i]);
    }
    r.sort((a, b) => a[0] - b[0]);

    let bw = 0;
    let vw = [];
    for (let w of walls) {
        if (rs.has(w)) bw++;
        else vw.push(w);
    }
    vw.sort((a, b) => a - b);

    const lowerBound = (arr, val) => {
        let l = 0, right = arr.length;
        while (l < right) {
            let mid = (l + right) >> 1;
            if (arr[mid] >= val) right = mid;
            else l = mid + 1;
        }
        return l;
    };

    const upperBound = (arr, val) => {
        let l = 0, right = arr.length;
        while (l < right) {
            let mid = (l + right) >> 1;
```

```
            if (arr[mid] > val) right = mid;
            else l = mid + 1;
        }
        return l;
    };

    const getW = (x, y) => {
        if (x > y) return 0;
        let i1 = lowerBound(vw, x);
        let i2 = upperBound(vw, y);
        return i2 - i1;
    };

    let dp = Array.from({length: n}, () => [0, 0]);
    dp[0][0] = getW(r[0][0] - r[0][1], r[0][0] - 1);
    dp[0][1] = 0;

    for (let i = 1; i < n; i++) {
        let L = r[i-1][0], R = r[i][0];
        let d1 = r[i-1][1], d2 = r[i][1];

        let eR = Math.min(R - 1, L + d1);
        let sL = Math.max(L + 1, R - d2);

        let wR = getW(L + 1, eR);
        let wL = getW(sL, R - 1);
        let wBoth = (eR >= sL) ? getW(L + 1, R - 1) : wR + wL;

        dp[i][0] = Math.max(dp[i-1][0] + wL, dp[i-1][1] + wBoth);
        dp[i][1] = Math.max(dp[i-1][0], dp[i-1][1] + wR);
    }

    let wEnd = getW(r[n-1][0] + 1, r[n-1][0] + r[n-1][1]);
    return Math.max(dp[n-1][0], dp[n-1][1] + wEnd) + bw;
};
```

Java:

```
class Solution {
    public int maxWalls(int[] robots, int[] distance, int[] walls) {
        int n = robots.length;

        // Store robots as {position, distance}
        int[][] x = new int[n + 1][2];
        for (int i = 0; i < n; i++) {
```

```
    x[i][0] = robots[i];
    x[i][1] = distance[i];
}

// Sort robots by position
Arrays.sort(x, 0, n, (a, b) -> a[0] - b[0]);

// Sort walls
Arrays.sort(walls);

// Dummy robot
x[n][0] = (int)1e9;
x[n][1] = 0;

// Binary search helpers
// lower_bound
java.util.function.BiFunction<int[], Integer, Integer> lowerBound = (arr, target) -> {
    int l = 0, r = arr.length;
    while (l < r) {
        int mid = (l + r) / 2;
        if (arr[mid] < target) l = mid + 1;
        else r = mid;
    }
    return l;
};

// upper_bound
java.util.function.BiFunction<int[], Integer, Integer> upperBound = (arr, target) -> {
    int l = 0, r = arr.length;
    while (l < r) {
        int mid = (l + r) / 2;
        if (arr[mid] <= target) l = mid + 1;
        else r = mid;
    }
    return l;
};

// Function to count walls in [l, r]
java.util.function.BiFunction<Integer, Integer, Integer> query = (l, r) -> {
    if (l > r) return 0;
    int it1 = upperBound.apply(walls, r);
    int it2 = lowerBound.apply(walls, l);
    return it1 - it2;
};
```

```
// dp[i][0] = shoot LEFT
// dp[i][1] = shoot RIGHT
int[][] dp = new int[n][2];

// Base case
dp[0][0] = query.apply(x[0][0] - x[0][1], x[0][0]);

if (n > 1) {
    dp[0][1] = query.apply(
        x[0][0],
        Math.min(x[1][0] - 1, x[0][0] + x[0][1])
    );
} else {
    dp[0][1] = query.apply(x[0][0], x[0][0] + x[0][1]);
}

// DP transitions
for (int i = 1; i < n; i++) {

    // Case 1: shoot RIGHT
    dp[i][1] = Math.max(dp[i - 1][0], dp[i - 1][1]) +
            query.apply(
                x[i][0],
                Math.min(x[i + 1][0] - 1, x[i][0] + x[i][1])
            );

    // Case 2: shoot LEFT (no overlap)
    dp[i][0] = dp[i - 1][0] +
            query.apply(
                Math.max(x[i][0] - x[i][1], x[i - 1][0] + 1),
                x[i][0]
            );

    // Case 3: shoot LEFT with overlap handling
    int leftStart = Math.max(x[i][0] - x[i][1], x[i - 1][0] + 1);
    int leftEnd = x[i][0];

    int overlapStart = leftStart;
    int overlapEnd = Math.min(x[i - 1][0] + x[i - 1][1], x[i][0] - 1);

    int res = dp[i - 1][1]
            + query.apply(leftStart, leftEnd)
            - query.apply(overlapStart, overlapEnd);
```

```
            dp[i][0] = Math.max(dp[i][0], res);
        }

        return Math.max(dp[n - 1][0], dp[n - 1][1]);
    }
}
```